



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Ingegneria
“Enzo Ferrari”

Fondamenti di Elettronica Digitale

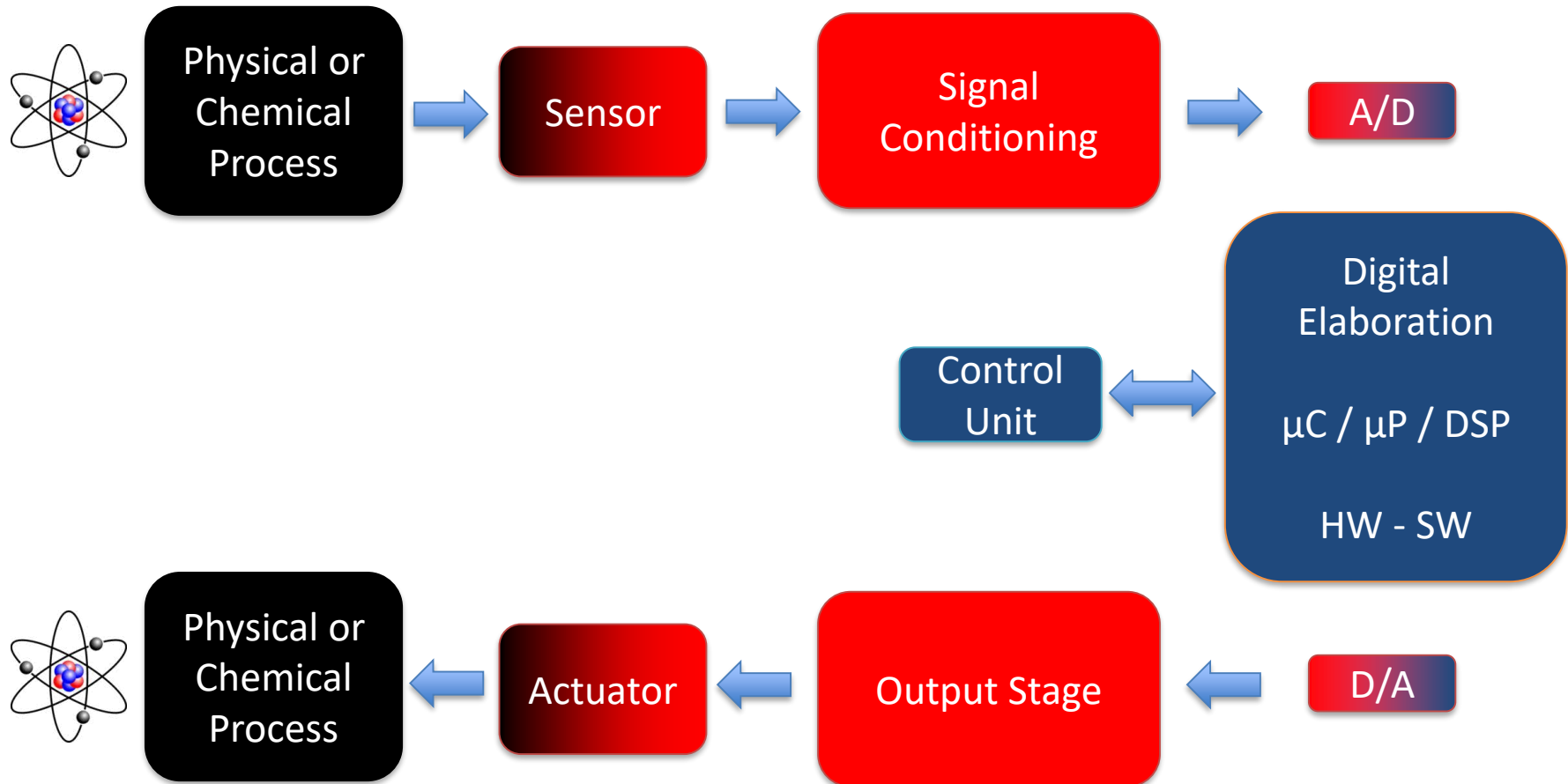
Digital Electronics

Laurea Triennale in Ingegneria Informatica

Prof. Francesco Maria Puglisi
email: francescomaria.puglisi@unimore.it
phone: +39-059-2056352

In the Digital (Sub-)system

Cheap and Fast Computation with 0s and 1s



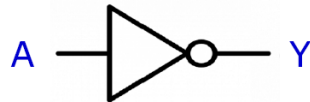
Recap on Digital Electronics

Logic and Digital data

Fundamental Digital Gates

Tabella della verità

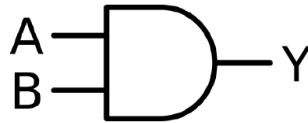
A	Y
0	1
1	0



Negazione logica
("Inverter")

Tabella della verità

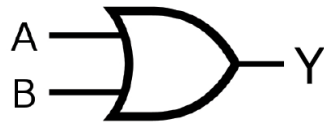
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



Prodotto logico
(porta **AND**)

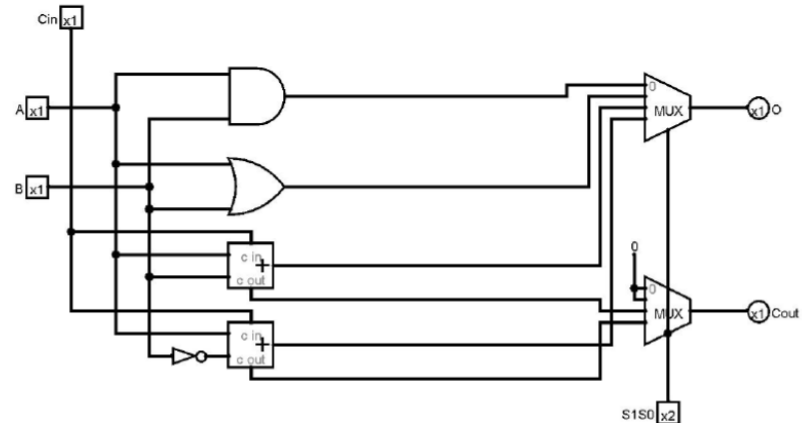
Tabella della verità

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

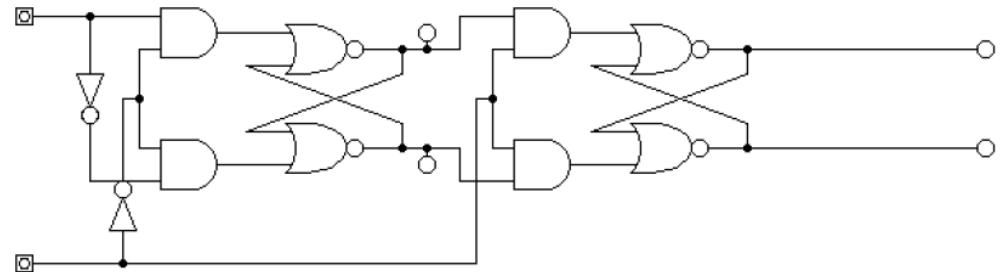


Somma logica
(porta **OR**)

Combinatorial circuits (example ALU)



Sequential circuits (example FSM, Flip-Flops)



Arithmetic Logic Unit

0s and 1s

An arithmetic logic unit (ALU) is a combinational digital electronic circuit that performs arithmetic and bitwise operations on integer binary numbers. Core of microprocessors CPU, GPU, FPU (floating point).

The inputs to an ALU are:

- the data (operands)
- a code indicating the operation to be performed
- a status with information about the previous operation

The outputs of an ALU are:

- the result
- a status with information about the previous operation

Typical operations:

Arithmetic: Add (w/ carry), Subtract (w/ borrow), 2's complement (negate), Increment, Decrement, Pass

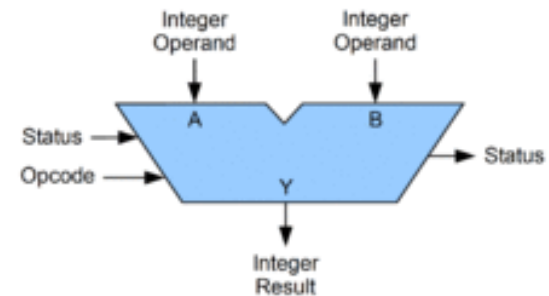
Bitwise: AND, OR, XOR, NOT

Arithmetic shift: input as 2's complement integer, MSB ("sign" bit) is preserved.

Logical shift: a logic zero is shifted into the operand. This is used to shift unsigned integers.

Rotate: a shift in a circular buffer fashion (MSB and LSB are adjacent).

Rotate through carry: the carry bit and operand are collectively treated as a circular buffer of bits.

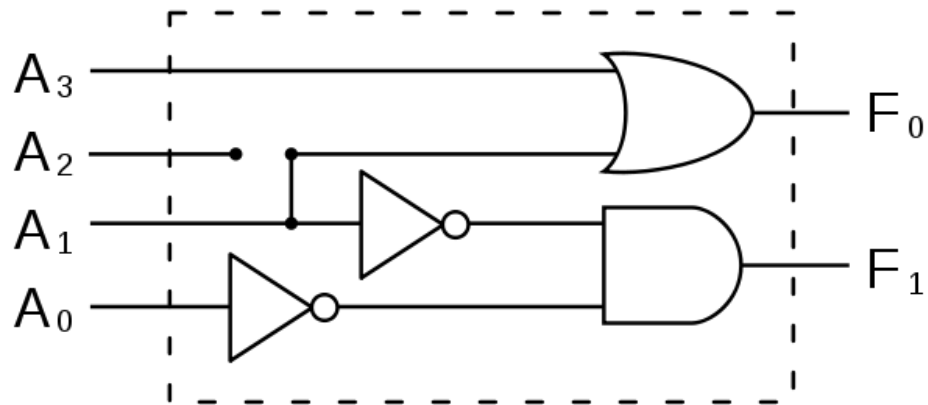


Encoder

A useful digital circuit

An encoder is a one-hot to binary converter. That is, if there are 2^n input lines, and **at most only one of them will ever be high**, the binary code of this 'hot' line is produced on the n-bit output lines.

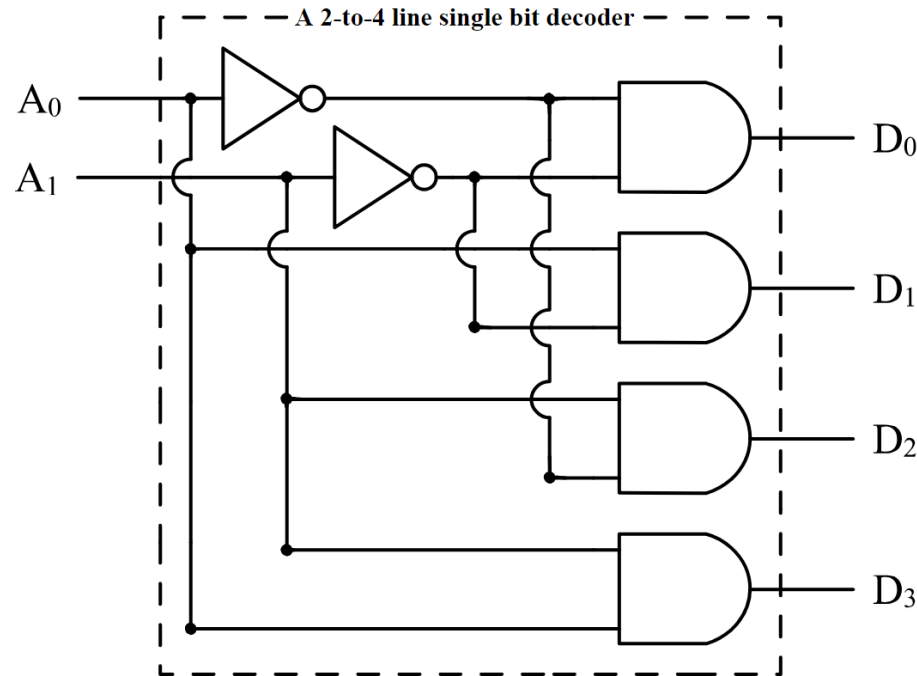
For example, a 4-to-2 simple encoder takes 4 input bits and produces 2 output bits. For all the non-explicitly defined input combinations (i.e., inputs containing > 1 high bits) the outputs are treated as don't cares. If the input circuit cannot guarantee at most a single-active input, this circuit can generate the wrong output when there is more than one input present in a high state. To rectify this problem a Priority encoder is used.



Decoder

And its counterpart

A binary decoder is a combinational logic circuit that converts binary information from the n -coded inputs to a maximum of 2^n **unique** outputs. They are used in a wide variety of applications, including data de-multiplexing, seven segment displays, and memory address decoding.



Truth Table

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Minterm Equations

$$D_0 = \overline{A_1} \cdot \overline{A_0}$$

$$D_1 = \overline{A_1} \cdot A_0$$

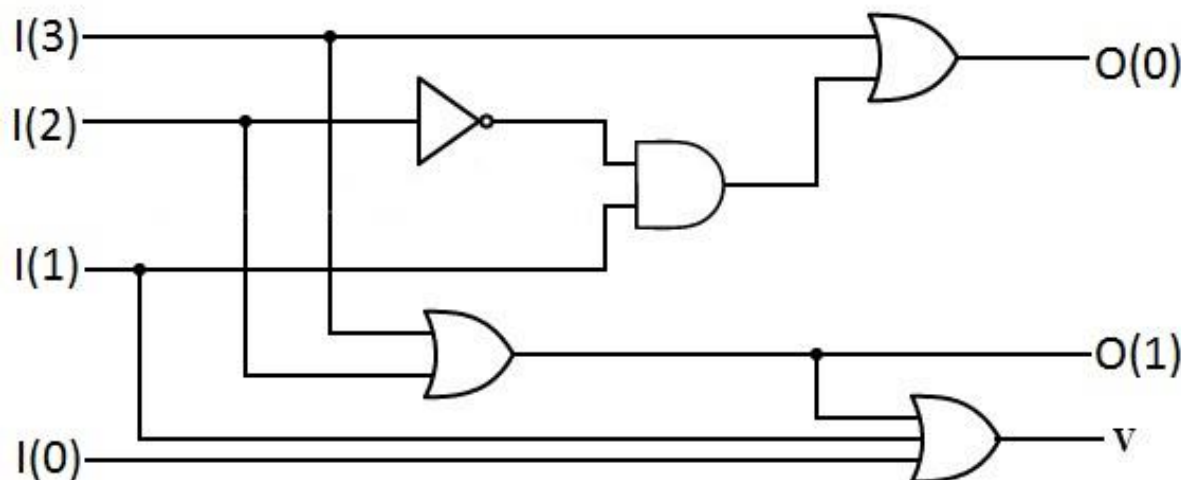
$$D_2 = A_1 \cdot \overline{A_0}$$

$$D_3 = A_1 \cdot A_0$$

Priority Encoder

Solving the simple encoder issue

If two or more inputs are given at the same time, the input having the highest priority will take precedence. In the diagram below (displaying a 4-to-2 PE), highest-priority inputs are to the top and "x" indicates an irrelevant value - i.e. any input value yields the same output since it is superseded by higher-priority input. The output V indicates if the input is valid. Priority encoders can be easily connected in arrays to make larger encoders, such as one 16-to-4 encoder made from six 4-to-2 priority encoders - four 4-to-2 encoders having the signal source connected to their inputs, and the two remaining encoders take the output of the first four as input.



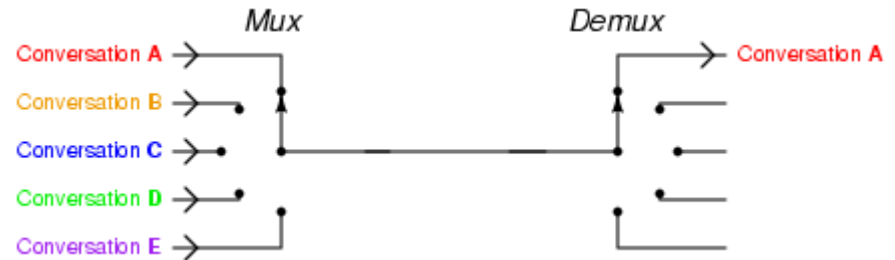
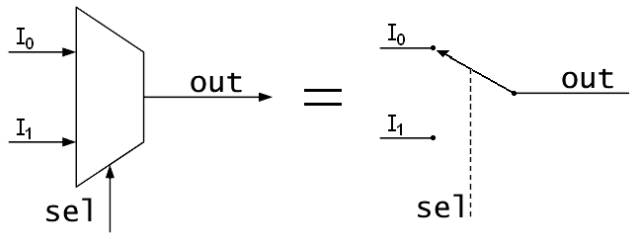
4 to 2 Priority Encoder

I ₃	I ₂	I ₁	I ₀	O ₁	O ₀	V
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

Multiplexer and De-multiplexer

Routing digital (and analog) signals

A multiplexer (or mux) is a device that selects one of several analog or digital input signals and forwards the selected input into a single line. A multiplexer of 2^n inputs has n select lines, which are used to select which input line to send to the output. A multiplexer makes it possible for several signals to share one device or resource, for example, one A/D converter or one communication line, instead of having one device per input signal.

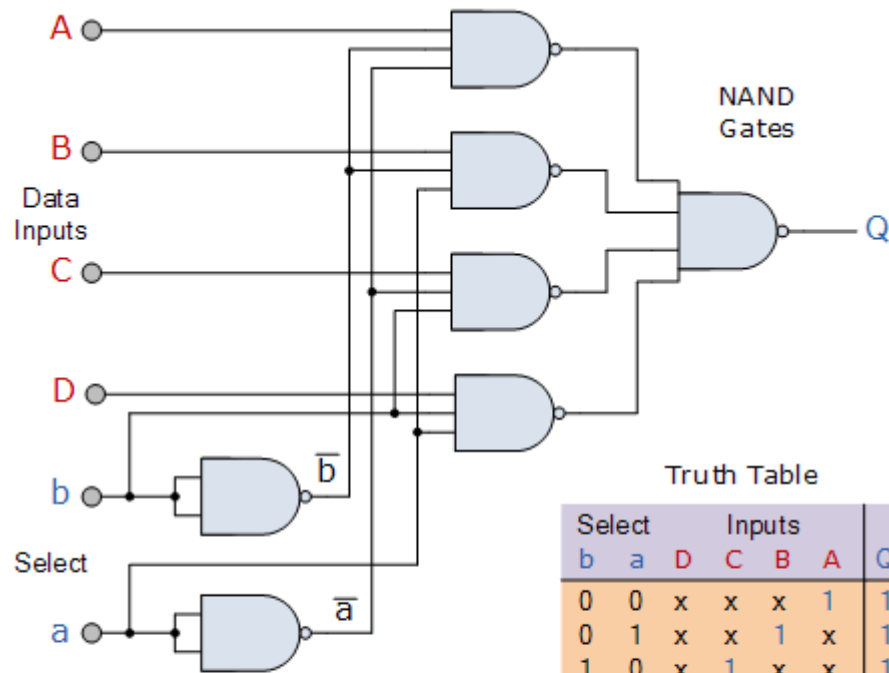


A de-multiplexer (or de-mux) is a device taking a single input signal and selecting one of many data-output-lines, which is connected to the single input. A multiplexer is often used with a complementary de-multiplexer on the receiving end.

Multiplexer and De-multiplexer

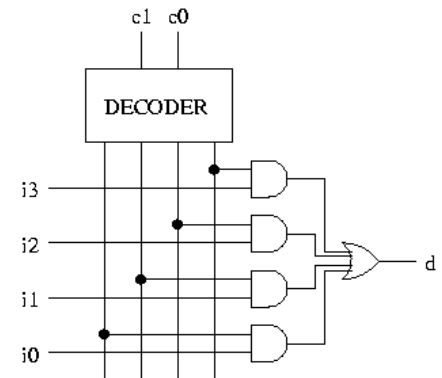
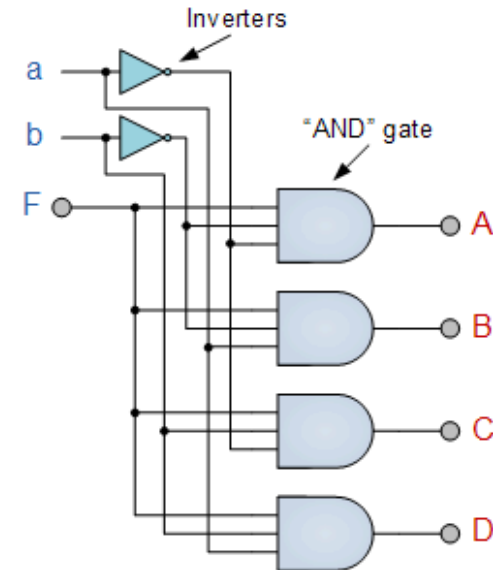
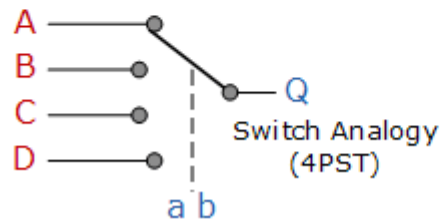
Routing digital (and analog) signals

An example of a 4-to-1 multiplexer with its de-mux counterpart is given below



Truth Table

Select			Inputs			
b	a	D	C	B	A	Q
0	0	x	x	x	1	1
0	1	x	x	1	x	1
1	0	x	1	x	x	1
1	1	1	x	x	x	1



An alternative
4-to-1 mux
implementation

Combinatorial and Sequential

A tale of two paradigms

It is possible to realize two kinds of logic circuits:

- Combinatorial
 - The output of a combinatorial circuit depends only on the input at present time (it is a memoryless circuit – the past does not matter)
- Sequential
 - The output of a sequential circuit depends on the input at present time and also at past times (the past matters)

That is, **sequential logic has state (memory)** while combinational logic does not.

To realize sequential circuits it is imperative to have a memory element!

In addition, to realize sequential circuits, it is better to somehow keep track of time (i.e., we need a **clock**, a signal that dictates timing!)

Clock

Synchronizing the circuit

It is important to have a reference signal that can somehow be used by the logic circuitry to guarantee that all the signals are stable and “make sense” when each logic operation is performed. This signal is called the clock.

Typically obtained on chip by using a quartz (very weak signal but very precise and stable oscillation frequency).

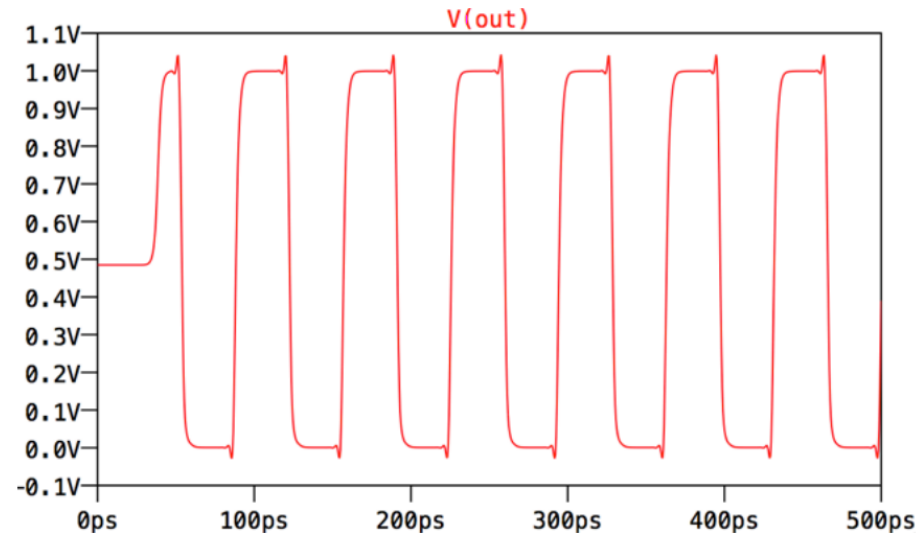
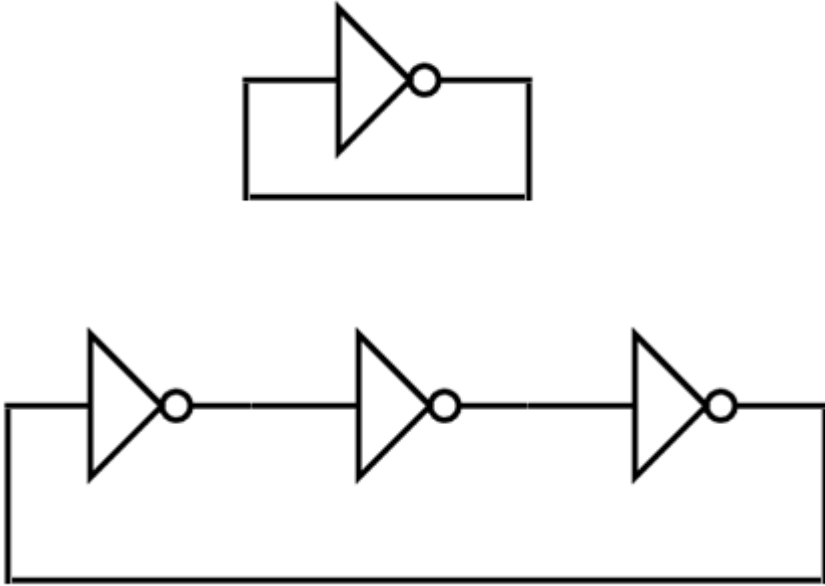
The clock is typically a square wave signal that tells the circuitry when to propagate the information to the next functional block. It acts on the edges of the signal (rising edge, falling edge, or both).

Ring Oscillator

Feedback in digital circuit

The simple NOT gate can be used to realize very interesting circuits.

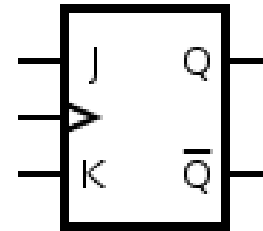
An example is the ring oscillator, a real square wave generator based on a simple looped chain of an **odd** number of NOT gates.



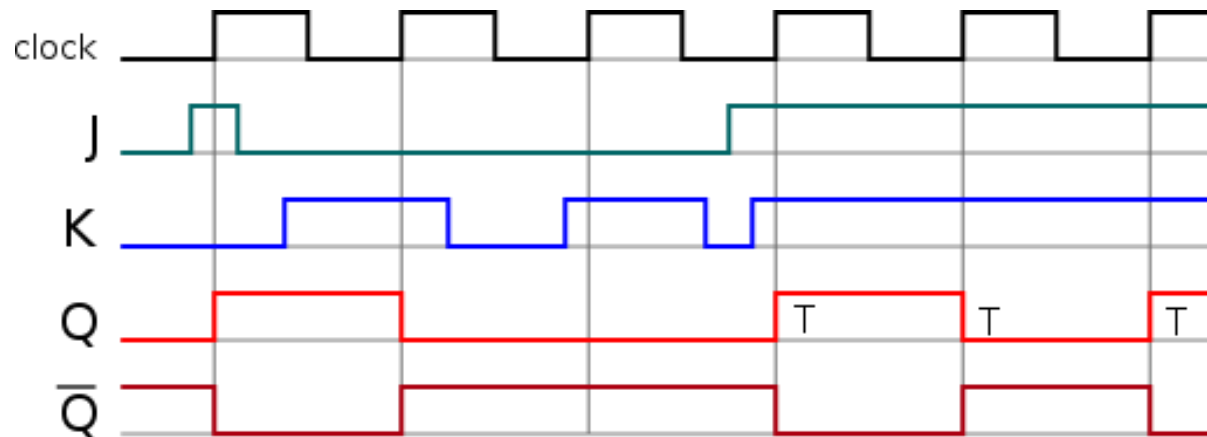
Flip-Flops

Storing digital data

The JK flip-flop augments the behavior of the SR flip-flop ($J=\text{Set}$, $K=\text{Reset}$) by interpreting the $J = K = 1$ condition as a "flip" or toggle command. To synthesize a D flip-flop, simply set K equal to the complement of J . Similarly, to synthesize a T flip-flop, set K equal to J . The JK flip-flop is therefore a universal flip-flop, because it can be configured to work as an SR flip-flop, a D flip-flop, or a T flip-flop.



Characteristic table			
J	K	Comment	Q_{next}
0	0	hold state	Q
0	1	reset	0
1	0	set	1
1	1	toggle	$\bar{Q}$



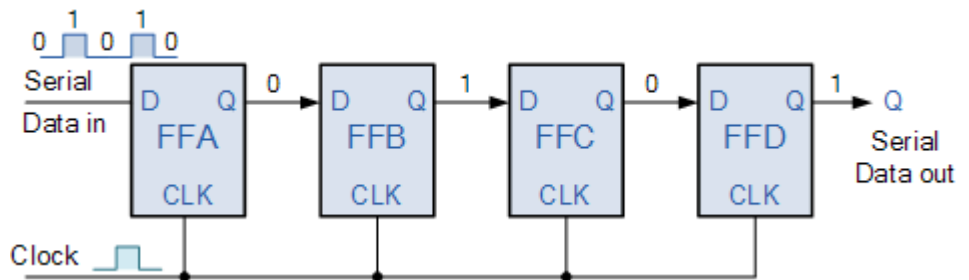
T = toggle

Shift Registers

Serial vs. Parallel

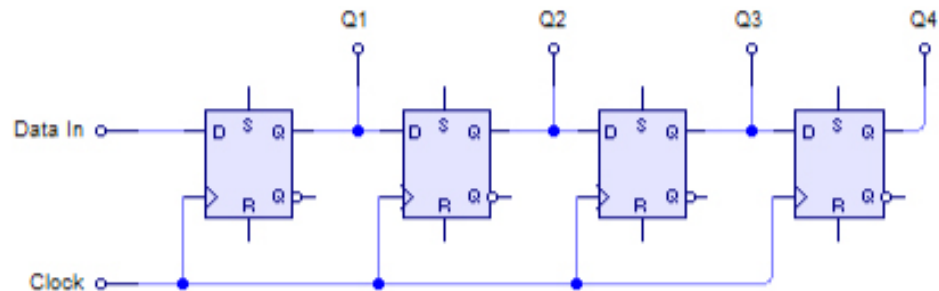
A shift register is **a cascade of flip flops**, sharing the same clock, in which the output of each flip-flop is connected to the data input of the next flip-flop, resulting in a circuit that shifts by one position the bit array stored in it, shifting in the data present at its input and shifting out the last bit in the array (that gets “lost”), at each transition of the clock input.

Serial-in serial-out (SISO)



Serial-in parallel-out (SIPO)

Can perform S-to-P conversion

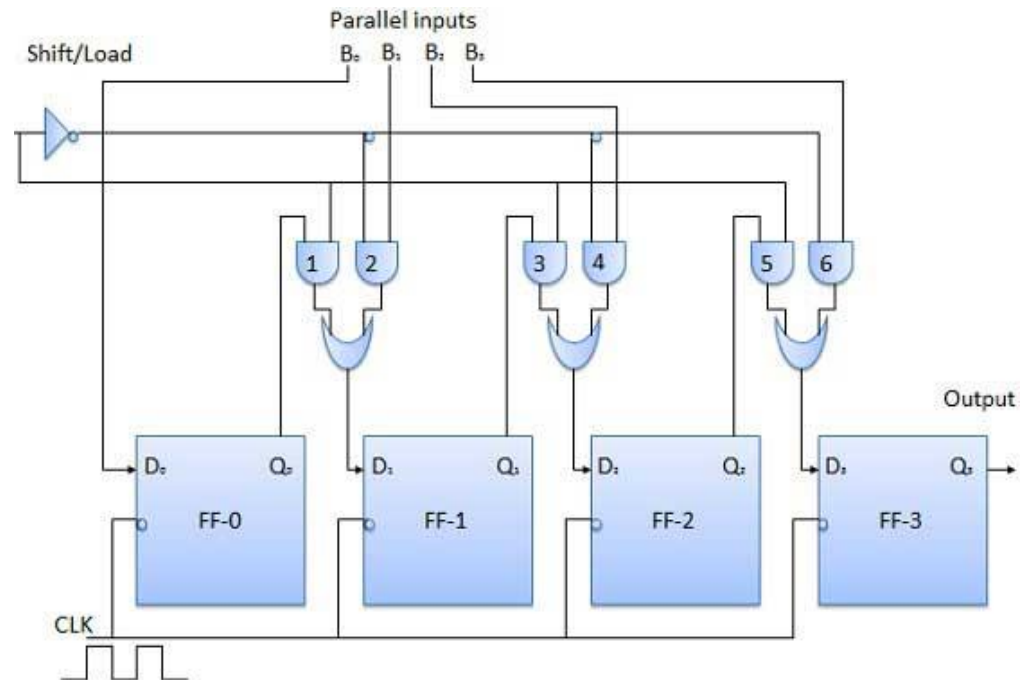


Shift Registers

Serial vs. Parallel

A shift register is **a cascade of flip flops**, sharing the same clock, in which the output of each flip-flop is connected to the data input of the next flip-flop, resulting in a circuit that shifts by one position the bit array stored in it, shifting in the data present at its input and shifting out the last bit in the array (that gets “lost”), at each transition of the clock input.

Parallel-in serial-out (PISO)
Can perform P-to-S conversion

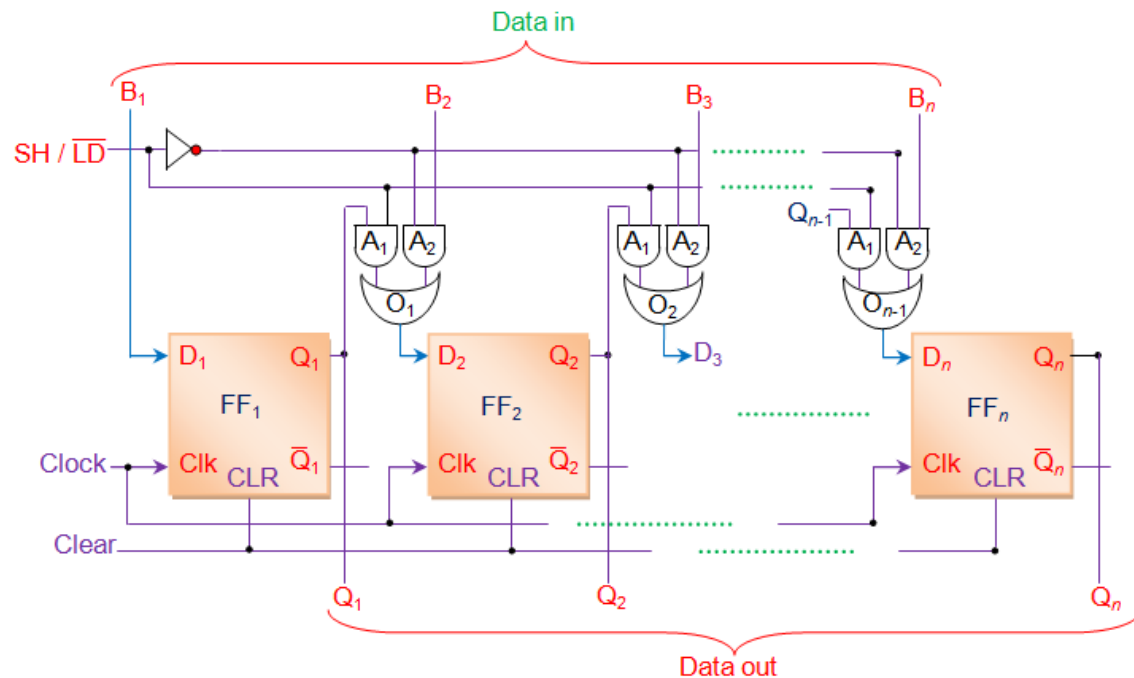


Shift Registers

Serial vs. Parallel

A shift register is **a cascade of flip flops**, sharing the same clock, in which the output of each flip-flop is connected to the data input of the next flip-flop, resulting in a circuit that shifts by one position the bit array stored in it, shifting in the data present at its input and shifting out the last bit in the array (that gets “lost”), at each transition of the clock input.

Parallel-in parallel-out (PIPO)

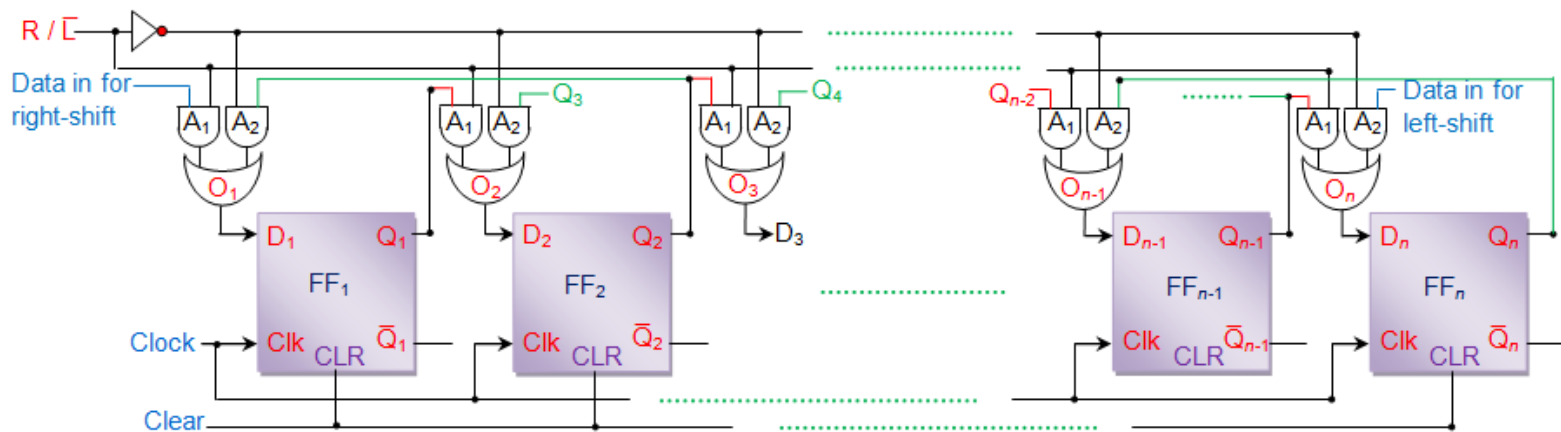


Shift Registers

Serial vs. Parallel

A shift register is **a cascade of flip flops**, sharing the same clock, in which the output of each flip-flop is connected to the data input of the next flip-flop, resulting in a circuit that shifts by one position the bit array stored in it, shifting in the data present at its input and shifting out the last bit in the array (that gets “lost”), at each transition of the clock input.

Bidirectional

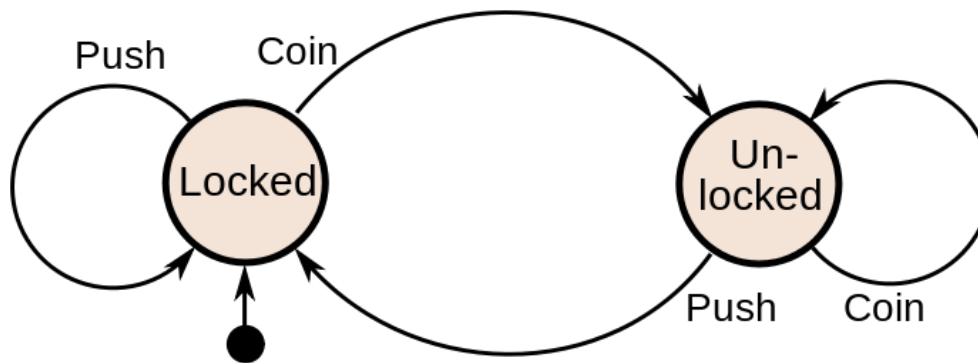


Finite State Machine

Automata

A finite-state machine (FSM) or finite-state automaton (FSA), or simply a state machine, is a mathematical model of computation. It is an abstract machine that **can be in exactly one of a finite number of states at any given time**. The FSM can change from one state to another in response to some external inputs; the change from one state to another is called a transition.

It is **defined by a list of its states, its initial state, and the conditions for each transition**

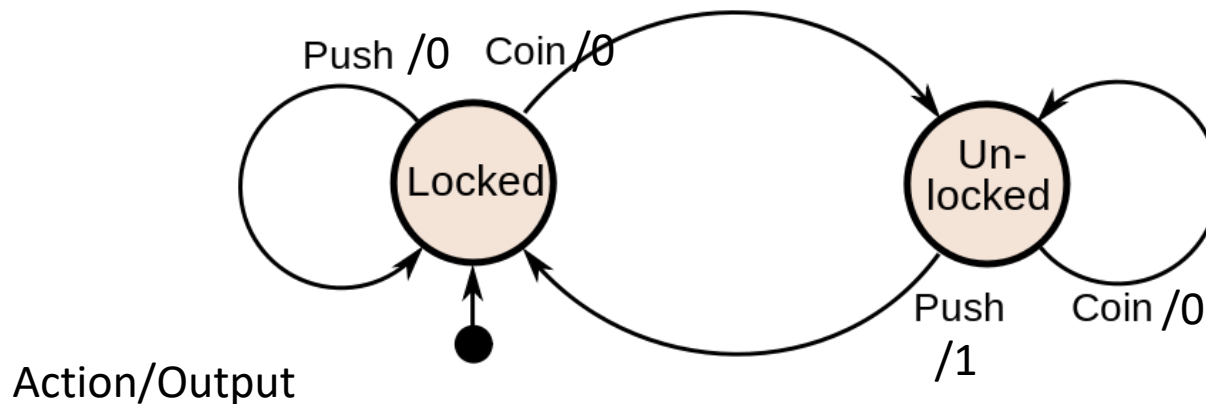


Finite State Machine

Automata

A finite-state machine (FSM) or finite-state automaton (FSA), or simply a state machine, is a mathematical model of computation. It is an abstract machine that **can be in exactly one of a finite number of states at any given time**. The FSM can change from one state to another in response to some external inputs; the change from one state to another is called a transition.

It is **defined by a list of its states, its initial state, and the conditions for each transition**

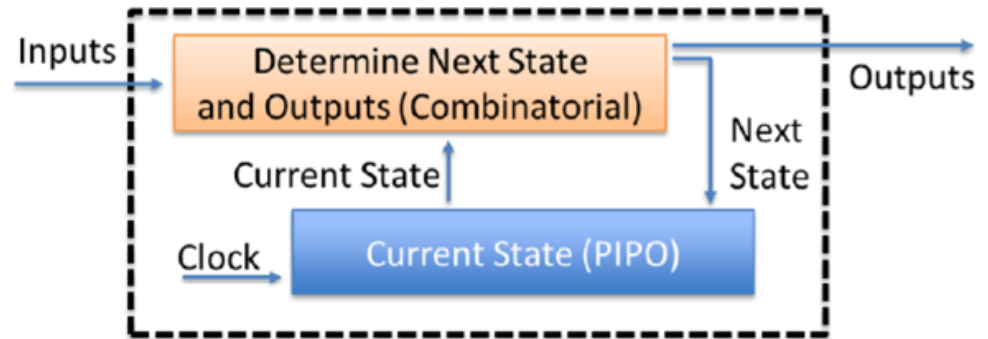
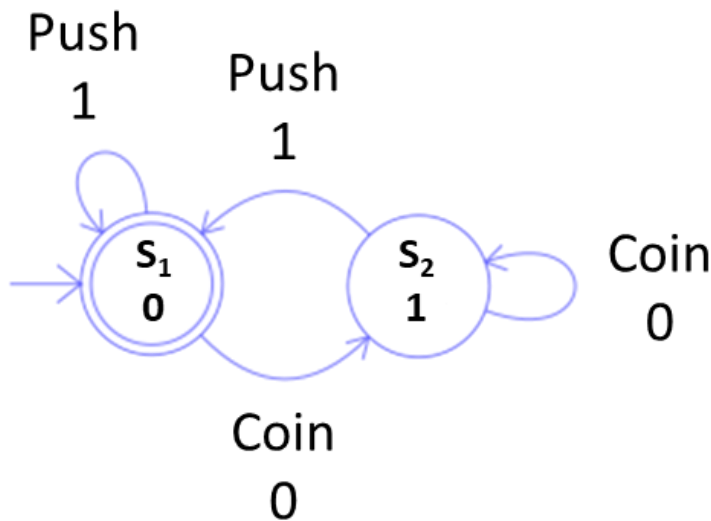


Finite State Machine

Automata

How can we build the corresponding logic circuit? (let's ignore the output)

Hint: we need a memory element to represent the current state and some combinatorial logic to determine transition rules (i.e., next state, function of the input and of the current state)



Input	Current State	Next State
0	0	1
1	0	0
0	1	1
1	1	0

Logic Families

Many flavors of the same logic

A logic family of digital integrated circuit devices is a group of electronic logic gates constructed using one of several different designs, usually with **compatible logic levels and power supply** characteristics within a family.

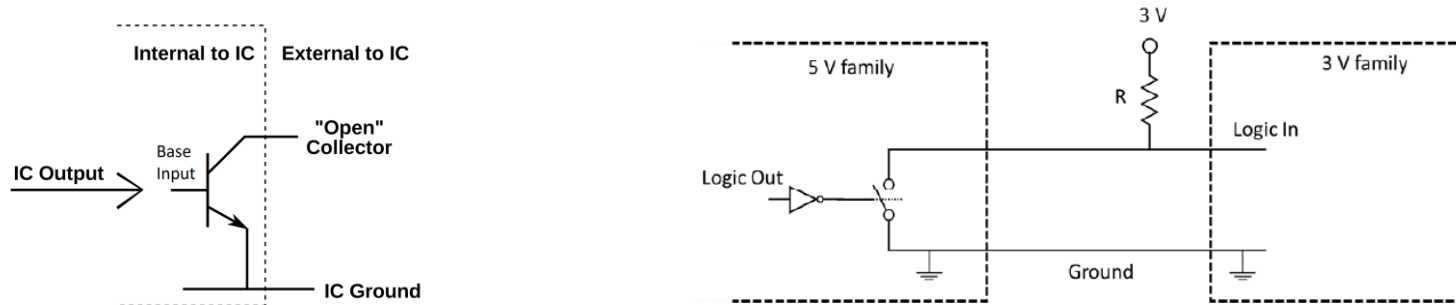
Famous examples: RTL, ECL, TTL, CMOS

Family ↕	Description ↕	Propagation delay (ns) ↕	Toggle speed (MHz) ↕	Power per gate @1 MHz (mW) ↕	Typical supply voltage V (range) ↕	Introduction year ↕	Remarks ↕
RTL	Resistor–transistor logic	500	4	10	3.3	1963	the first CPU built from integrated circuits (the Apollo Guidance Computer) used RTL.
DTL	Diode–transistor logic	25		10	5	1962	Introduced by Signetics, Fairchild 930 line became industry standard in 1964
CMOS	AC/ACT	3	125	0.5	3.3 or 5 (2-6 or 4.5-5.5)	1985	ACT has TTL compatible levels
CMOS	HC/HCT	9	50	0.5	5 (2-6 or 4.5-5.5)	1982	HCT has TTL compatible levels
CMOS	4000B/74C	30	5	1.2	10V (3-18)	1970	Approximately half speed and power at 5 volts
TTL	Original series	10	25	10	5 (4.75-5.25)	1964	Several manufacturers
TTL	L	33	3	1	5 (4.75-5.25)	1964	Low power
TTL	H	6	43	22	5 (4.75-5.25)	1964	High speed
TTL	S	3	100	19	5 (4.75-5.25)	1969	Schottky high speed
TTL	LS	10	40	2	5 (4.75-5.25)	1976	Low power Schottky high speed
TTL	ALS	4	50	1.3	5 (4.5-5.5)	1976	Advanced Low power Schottky
TTL	F	3.5	100	5.4	5 (4.75-5.25)	1979	Fast
TTL	AS	2	105	8	5 (4.5-5.5)	1980	Advanced Schottky
TTL	G	1.5	1125 (1.125 GHz)		1.65 - 3.6	2004	First GHz 7400 series logic
ECL	ECL III	1	500	60	-5.2(-5.19 - -5.21)	1968	Improved ECL
ECL	MECL I	8		31	-5.2	1962	first integrated logic circuit commercially produced
ECL	ECL 10K	2	125	25	-5.2(-5.19 - -5.21)	1971	Motorola
ECL	ECL 100K	0.75	350	40	-4.5(-4.2 - -5.2)	1981	
ECL	ECL 100KH	1	250	25	-5.2(-4.9 - -5.5)	1981	

Open Collector

Bringing them together

A type of output that is either put to ground or disconnected.



The pull-up resistor is external and is not connected to the chip supply voltage, so a different voltage than the chip supply voltage can be used. (Caveat!) Can be used to interface different families having different operating voltages.

More than one open-collector output can be connected to a single line. Open-collector devices are commonly used to connect multiple devices to a shared bus. This enables one device to drive the bus without interference from the other inactive devices.

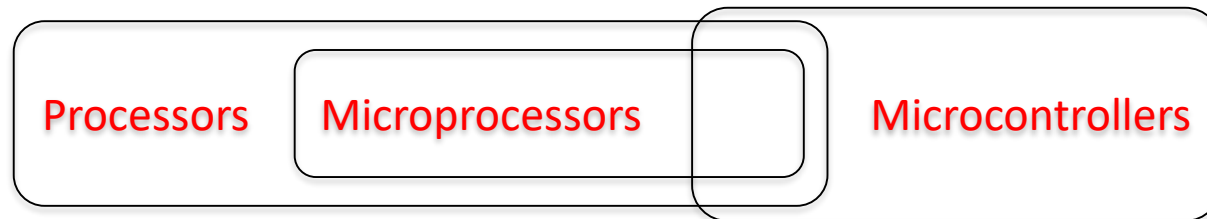
Microcontrollers

Digital systems

A **processor** is a hardware device conceived to **execute instructions**

A **microprocessor** is a kind of processor the hardware of which is all contained in an IC.

A **microcontroller** contains a microprocessor with a reduced instruction set (RISC) and some dedicated peripheral circuits.



Microcontrollers are designed for embedded applications, in contrast to the microprocessors used in PCs. Microcontrollers are used in automatically controlled products and devices, such as automobile engine control systems and other embedded systems. By reducing the size and cost compared to a design that uses a separate microprocessor, memory, and input/output devices, microcontrollers make it economical to digitally control even more devices and processes.

Digital systems

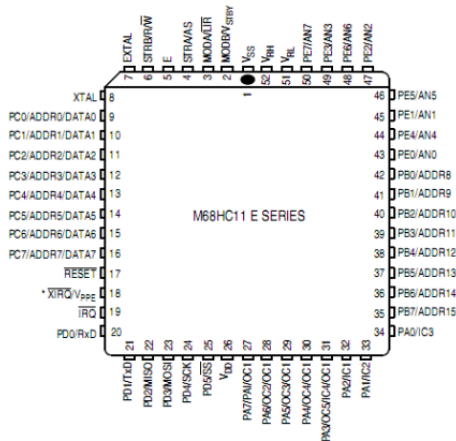
Typical input and output devices include switches, relays, solenoids, LED's, small or custom liquid-crystal displays, radio frequency devices, and sensors for data such as temperature, humidity... It typically integrates also A/D and/or D/A circuits and timers.



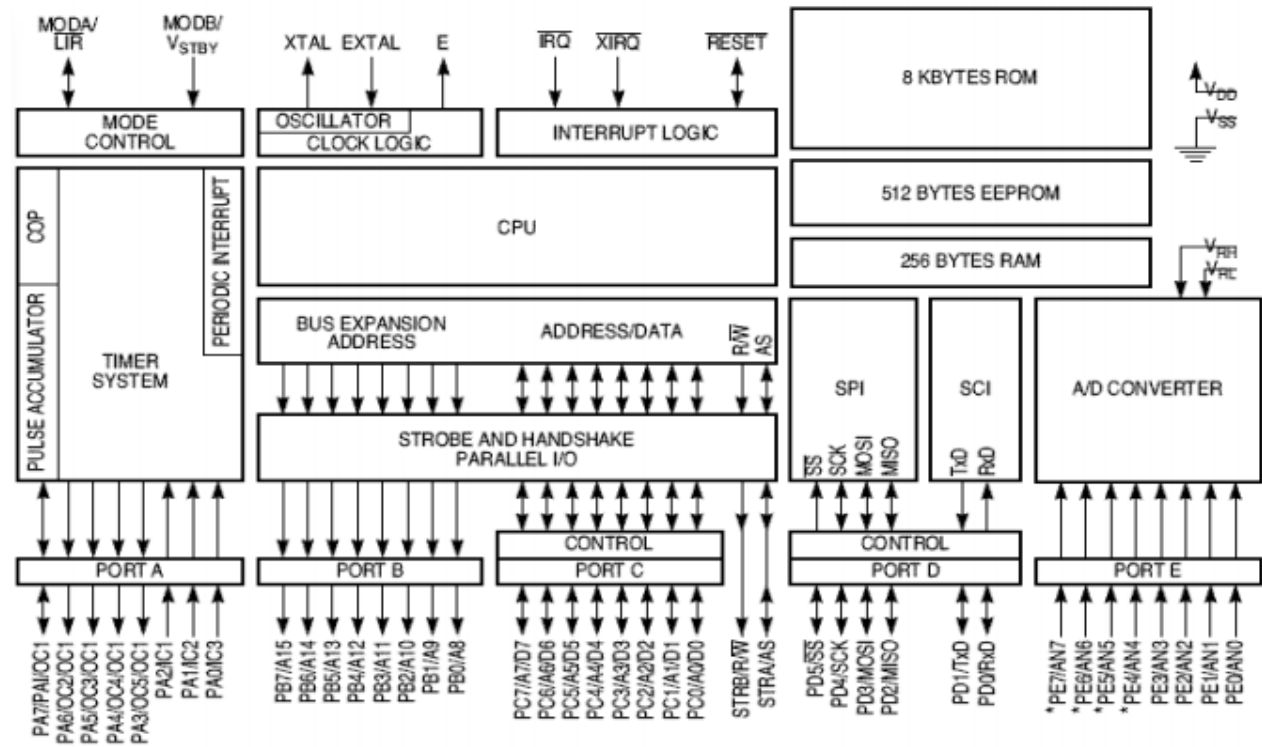
MOTOROLA
68HC11x Family



Approx 70 MCU
68HC11x



* V_{PP} applies only to devices with EPROM/OTPROM.



* NOT BONDED ON 48-PIN VERSION.

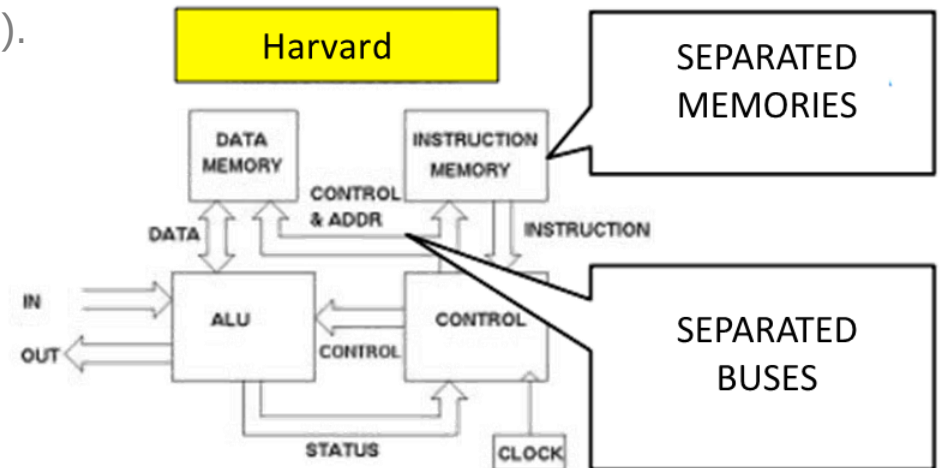
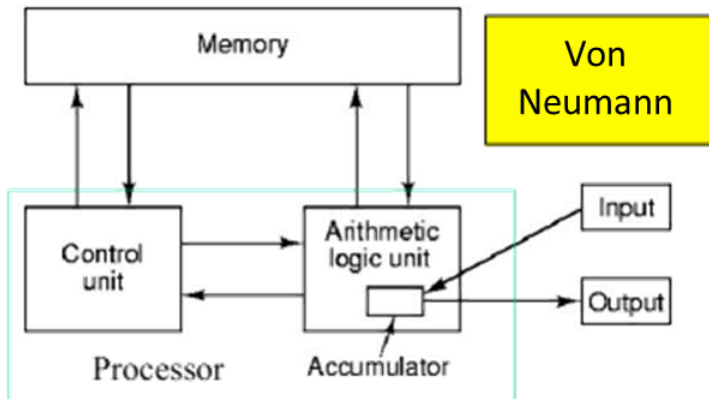
Microcontrollers

Digital systems

Typical architectures:

Von Neumann: Memory contains both instructions and operands (instruction memory and data memory). Data bus and control bus are shared. Cannot simultaneously fetch data and instructions (slower but simpler). Much more common nowadays that clock frequency is very high.

Harvard: Separate memories for instructions and operands (instruction memory and data memory). Data bus and control bus are separated. Can simultaneously fetch data and instructions (faster but more complicated).



Microcontrollers

Digital systems

Advantages:

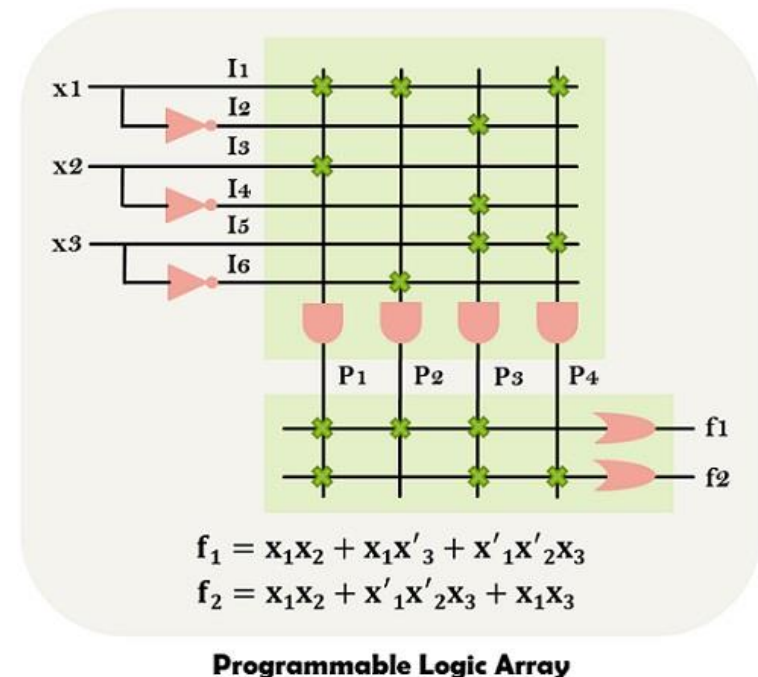
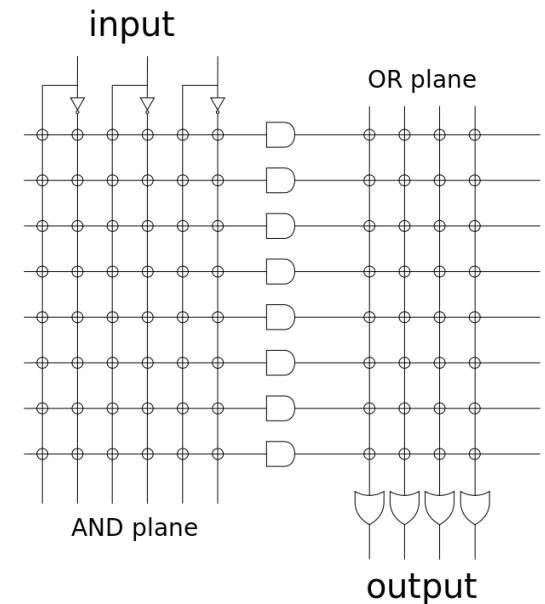
- Derived from μ P but using a reduced instruction set (simpler hardware)
- Allow a more specific use in industrial applications where many instructions of a μ P are typically not used
- Rapid evolution of their sophistication together with a strong cost reduction
- Can be optimized so that its execution speed for specific instructions is even higher than the one of μ P
- Smaller and less power hungry than a μ P
- Fewer components means higher reliability
- Compared to dedicated (application-specific) controllers they can execute instructions so they can:
 - Execute complex functions
 - Communicate with other devices
 - Be re-programmed! (by the way, how do you program a microcontroller?)
- Can have low-power working regimes, frequently programmable via software
- Available in many flavors: 4 to 32 bits, 1 to 4 cores, few kHz to few GHz, RISC/CISC

PLAs and FPGAs

Digital systems

A programmable logic array (PLA) is a kind of programmable logic device used to implement combinational logic circuits. The PLA has a set of **programmable** AND gate planes, which link to a set of **programmable** OR gate planes, which can then be conditionally complemented to produce an output. This layout allows for a large number of logic functions to be synthesized in the sum of products canonical forms. Typically they are **mask-programmable** (meaning that they can be programmed during manufacturing, but then there is no mean to re-program them).

Field-programmable PLAs are called **FPGAs**.



PLAs and FPGAs

Digital systems

A field-programmable gate array (FPGA) is an integrated circuit designed to be configured by a customer or a designer after manufacturing. The FPGA configuration is generally specified using a hardware description language (HDL).

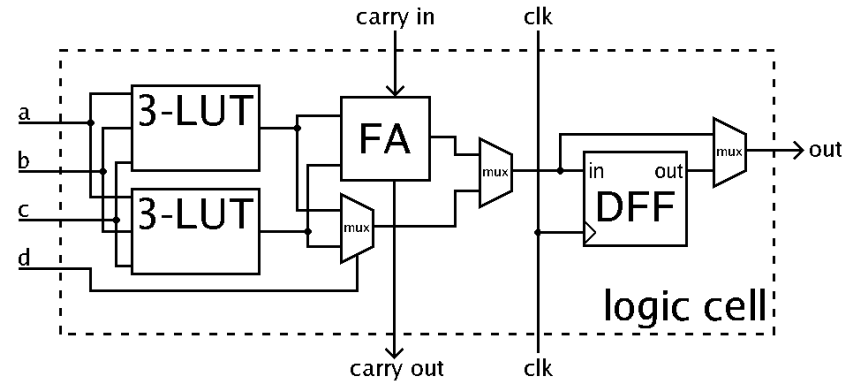
FPGAs contain an array of **programmable logic blocks**, and a hierarchy of **reconfigurable interconnects** that allow the blocks to be "wired together". Logic blocks can be configured to perform complex combinational functions. In most FPGAs, logic blocks also include memory elements, which may be simple flip-flops or more complete blocks of memory. FPGAs allow flexible reconfigurable computing as performed in computer software.

PLAs and FPGAs

Digital systems

Structure of an FPGA:

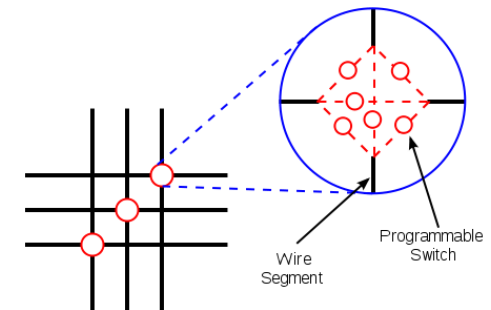
- Matrix of CLB (Configurable Logic Blocks)
 - They realize logic functions
- IOB (Input Output Blocks) at the edges of CLBs
 - Provide the interface with the external world
- Programmable interconnects
 - Provide smart reconfigurable connections



CLBs are made of two or four logic cells. Each LC has 1 or more LUTs (Look-up Tables), an SRAM register with a multiplexer, a FF and a Full Adder.

IOB can be programmed to handle different logic standards as TTL, CMOS, or PCI.

Programmable interconnections are realized by means of a switch matrix, that are networks of programmable transistor switches (their status is stored and loaded from a memory).



ASICs

Digital systems

An Application-Specific Integrated Circuit (ASIC), is an IC customized for a particular use, rather than intended for general-purpose use. Modern ASICs often include entire microprocessors, memory blocks including ROM, RAM, EEPROM, flash memory and other large building blocks. Such an ASIC is often termed a SoC (system-on-chip).

FPGAs are preferred to prototype from standard parts. For smaller designs or lower production volumes, FPGAs may be more cost effective than an ASIC design, even in production. Device manufacturers typically prefer FPGAs for prototyping and devices with low production volume and ASICs for very large production volumes where costs can be amortized across many devices.

Full-custom ASIC design defines all the layers of the device. The benefits of full-custom design include reduced area (and therefore recurring component cost), performance improvements, and also the ability to integrate analog components and other pre-designed components. The disadvantages include increased manufacturing and design time, increased engineering costs, more complexity in the computer-aided design (CAD), and a much higher skill requirement on the part of the design team.